

COMO RODAR E TESTAR (guia completo)

Roteiro para colocar o backend no ar do zero e testar todas as rotas. Os comandos foram escritos para **Windows + XAMPP + PowerShell** (o terminal padrão do VS Code), que é o ambiente usado no desenvolvimento.

Observação sobre o PowerShell: o PowerShell **não aceita** o operador `<` para enviar arquivos `.sql` ao MySQL (dá erro `The '<' operator is reserved`). Por isso, neste guia usamos `Get-Content arquivo.sql | mysql`. Se você usar o **Prompt de Comando (cmd)** em vez do PowerShell, aí pode usar `mysql < arquivo.sql`.

Pré-requisitos

- PHP 8.0 ou superior
- Composer
- MySQL 8.0+ (ou MariaDB) — vem junto no XAMPP

Confira o que você tem (no PowerShell):

```
php --version
composer --version
```

Para o MySQL, como ele fica dentro do XAMPP, confirme assim:

```
Test-Path C:\xampp\mysql\bin\mysql.exe
```

Se responder `True`, o MySQL do XAMPP está instalado.

PASSO 1 — Iniciar o XAMPP

1. Abra o **XAMPP Control Panel**.
2. Clique em **Start** nas linhas **Apache** e **MySQL** (devem ficar verdes).

O MySQL do XAMPP usa usuário `root` com **senha vazia** — é o que o `.env` do projeto já espera.

PASSO 2 — Ativar as extensões do PHP

O backend precisa de duas extensões do PHP ativadas: **zip** (para o Composer baixar pacotes) e **pdo_mysql** (para conectar no banco). Se você usa o PHP do XAMPP, normalmente já vêm ativas. Se usa um PHP instalado à parte (ex.: em `C:\php`), confira:

```
php -m
```

Procure por `zip` e `pdo_mysql` na lista.

Se faltar alguma, abra o `php.ini` do seu PHP (o caminho aparece em `php --ini`), localize a linha correspondente e remova o `;` do início:

```
;extension=zip      -> extension=zip
;extension=pdo_mysql -> extension=pdo_mysql
```

Salve e rode `php -m` de novo para confirmar.

PASSO 3 — Instalar as dependências

Dentro da pasta do projeto:

```
composer install
```

Isso cria a pasta `vendor/` (necessária — sem ela o login não funciona, pois a biblioteca do JWT mora ali).

Se aparecer um aviso de *security advisory* sobre `firebase/php-jwt`, confirme que o `composer.json` exige a versão `^7.0` (a versão 6 tinha uma vulnerabilidade conhecida e é bloqueada pelo Composer).

PASSO 4 — Conferir o `.env`

O arquivo `.env` já vem pronto para uso local (usuário `root`, sem senha). Confira/edite se necessário:

```
DB_HOST=localhost
DB_PORT=3306
DB_NAME=controle_estoque_pedidos
DB_USER=root
DB_PASS=

JWT_SECRET=coloque_uma_string_aleatoria_longa_aqui
JWT_EXPIRES=3600
```

JWT_SECRET: troque por uma string aleatória longa. Você pode gerar uma com: `powershell php -r "echo bin2hex(random_bytes(32));"` Copie o resultado e cole no `JWT_SECRET`. (O `.env` não vai para o GitHub.)

Se "Falha na conexão com o banco": tente trocar `DB_HOST=localhost` por `DB_HOST=127.0.0.1`.

PASSO 5 — Criar o banco de dados (15 tabelas)

O banco tem **15 tabelas**, criadas por migrations numeradas. A ordem importa por causa das chaves estrangeiras (uma tabela só pode referenciar outra que já exista).

Opção A — PowerShell (recomendada)

Cole o bloco abaixo **inteiro** no PowerShell, dentro da pasta do projeto. Ele roda todas as migrations na ordem certa e depois o seed:

```
$mysql = "C:\xampp\mysql\bin\mysql.exe"
$ordem = @(
    "00_create_database",
    "01_create_usuarios_table",
    "02_create_clientes_table",
    "03_create_produtos_table",
    "04_create_pedidos_table",
    "06_create_fornecedores_table",
    "05_create_itens_pedido_table",
    "07_create_historico_status_table",
    "08_create_enderecos_table",
    "09_create_materiais_table",
    "10_create_compras_table",
    "11_create_compra_material_table",
```

```

"12_create_produto_material_table",
"13_create_caixa_table",
"14_create_receitas_table",
"15_create_despesas_table"
)
foreach ($f in $ordem) {
  Write-Host "Rodando $f..."
  Get-Content "database/migrations/$f.sql" | & $mysql -u root
}
Write-Host "Rodando seed..."
Get-Content "database/seeds/dados_iniciais.sql" | & $mysql -u root

```

Por que a 06 vem antes da 05? A migration 08 (endereço) referencia a tabela `fornecedor` (06), então os fornecedores precisam existir antes. A 05 (item_pedido) só depende de pedido e produto, que já foram criados.

Se `mysql` não é reconhecido, use o caminho completo `C:\xampp\mysql\bin\mysql.exe` (já está na variável `$mysql` acima).

Opção B — phpMyAdmin (visual)

1. Acesse <http://localhost/phpmyadmin>
2. Vá em **Importar**.
3. Importe cada arquivo de `database/migrations/` na ordem da lista acima (00, 01, 02, 03, 04, 06, 05, 07, 08, 09, 10, 11, 12, 13, 14, 15).
4. Por último, importe `database/seeds/dados_iniciais.sql`.

Conferir se deu certo

```
& "C:\xampp\mysql\bin\mysql.exe" -u root -e "USE controle_estoque_pedidos; SHOW TABLES;"
```

Devem aparecer as **15 tabelas**: caixa, cliente, compra, compra_material, despesa, endereço, fornecedor, historico_status, item_pedido, material, pedido, produto, produto_material, receita, usuario.

Erro "Duplicate entry 'admin@local'" ao rodar o seed mais de uma vez é normal: significa que o usuário admin já existe. Pode ignorar.

PASSO 6 — Subir o servidor

Na pasta do projeto:

```
php -S localhost:8000 -t public
```

Deve aparecer `Development Server (http://localhost:8000) started`. Deixe essa janela **aberta** (é o servidor rodando). Para parar: `Ctrl+C`.

Teste rápido no navegador: <http://localhost:8000/api/produtos> Deve aparecer um JSON `"Token de autenticação não fornecido"` (401) — isso é **bom**: o servidor está no ar e a proteção das rotas funciona. (Pelo navegador sempre dá 401, porque ele não envia o token; o token só vai pelo Insomnia.)

PASSO 7 — Testar no Insomnia

1. Baixe o Insomnia: <https://insomnia.rest/download>
2. **Import** → **From File** → selecione `insomnia_collection.json`.

3. Faça o **Login** (pasta "1. Autenticação"): já vem com `admin@local` / `admin123` . Clique em **Send**. A resposta traz `dados.token` .
4. **Copie o token** (o texto longo, sem aspas) e cole na variável `token` : clique em **Base Environment** (canto superior esquerdo) e preencha `"token": "cole_aqui"` .
5. Agora as rotas protegidas funcionam.

Ordem recomendada de teste (respeita as dependências)

1. **Materiais → Criar** (cria um material)
2. **Compras → Criar** (registra compra; soma ao estoque dos materiais)
3. **Materiais → Listar** (confirme que o `estoque` aumentou)
4. **Pedidos → Criar** (cria pedido; baixa estoque do produto; grava histórico)
5. **Pedidos → Detalhar** (vem com `itens` e `historico`)
6. **Pedidos → Mudar status** e **Detalhar** de novo (o `historico` ganha outra linha)
7. **Receitas / Despesas → Criar** (entradas e saídas ligadas ao caixa)

O token expira em 1 hora (`JWT_EXPIRES=3600`). Quando as rotas começarem a dar 401, refaça o Login e atualize o token no Base Environment.

SE DER ERRADO (problemas comuns)

Sintoma	Causa provável	Solução
<code>The '<' operator is reserved</code>	Usou <code><</code> no PowerShell	Use <code>Get-Content</code> <code>arq.sql</code> <code> </code> <code>mysql</code> (ver Passo 5)
<code>mysql</code> não é reconhecido	MySQL não está no PATH	Use <code>C:\xampp\mysql\bin\mysql.exe</code>
Falha na conexão com o banco	Falta <code>pdo_mysql</code> ou host	Ative <code>pdo_mysql</code> (Passo 2); tente <code>127.0.0.1</code> no <code>.env</code>
<code>The zip extension ... missing</code>	Falta <code>zip</code> no PHP	Ative <code>extension=zip</code> (Passo 2)
<code>Class "...JWT" not found</code>	Faltou <code>composer install</code>	Rode o Passo 3
401 em todas as rotas	Token não colado/expirado	Refaça o Login e cole o token (validade: 1h)
Página/JSON em branco	Servidor sem <code>-t public</code>	Suba com <code>php -S localhost:8000 -t public</code>
<code>Cannot add foreign key</code> numa migration	Ordem errada das migrations	Siga a ordem do Passo 5 (06 antes da 05)
Porta 8000 ocupada	Outro processo usando	Use <code>php -S localhost:8080 -t public</code>

ESTRUTURA DO BANCO (15 tabelas)

Núcleo / acesso: usuario **Comercial:** cliente, endereco, pedido, item_pedido, produto, historico_status **Estoque:** fornecedor, material, compra, compra_material **Composição:** produto_material (quais materiais um produto usa)
Financeiro: caixa, receita, despesa

Relacionamentos que viram tabela associativa (muitos-para-muitos): - `compra_material` — uma compra contém vários materiais - `produto_material` — um produto utiliza vários materiais

CHECKLIST FINAL

- [] `composer install` rodou e criou `vendor/`
- [] Banco `controle_estoque_pedidos` criado com as **15 tabelas**
- [] Servidor sobe sem erro
- [] Login retorna token no Insomnia
- [] CRUD de produtos/materiais funciona
- [] Criar compra soma ao estoque do material
- [] Criar pedido baixa estoque e gera histórico de status
- [] Projeto subido no GitHub com README
- [] Arquivo TXT do SIGAA (link do repo + hash do commit + integrantes)